

Proyecto Único: Sistema Web de Monitoreo, Enrutamiento y Simulación de Conexiones Satelitales Distribuidas en Red Local (OrbitNet-NetCore)

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas (ECYS)
Departamento de Desarrollo de Software
Curso: Introducción a la Programación y Computación II
Vigencia: Primer Semestre / Escuela de Vacaciones 2026
Catedrático: Ing. Jaime Francisco Yumán Ramírez
Auxiliar de Cátedra: Fernando José Vicente Velásquez

1. Marco Formativo y Competencias Profesionales

El proceso de enseñanza de la Escuela de Ciencias y Sistemas de la Universidad de San Carlos de Guatemala exige una constante evolución en el diseño de sus prácticas de laboratorio. Para responder a las demandas del entorno tecnológico contemporáneo, la presente especificación técnica establece las bases de desarrollo del Proyecto Único para el laboratorio del curso Introducción a la Programación y Computación II. Este curso constituye la transición crítica entre el aprendizaje de la sintaxis estructurada básica y el dominio de estructuras de datos complejas no lineales que serán profundizadas en el curso de Estructura de Datos. El marco pedagógico de esta práctica sitúa al estudiante ante un escenario de ingeniería de software real que de forma obligatoria demanda la adopción estricta de metodologías de trabajo ágiles y control de versiones.

El ecosistema principal adoptado para esta práctica es C# y la plataforma .NET 8.0, sustituyendo esquemas interpretados previos para aproximar al estudiante al manejo estricto de tipos de datos, la gestión de la memoria dinámica controlada y el desarrollo de arquitecturas web bajo estándares empresariales. La administración del proyecto se rige por un conjunto inmutable de valores académicos estructurados para modelar el perfil profesional de los estudiantes:

Valor Académico	Descripción y Aplicación Práctica en el Proyecto
Disciplina	El estudiante administra y sigue el cronograma de entregas de forma rigurosa para cumplir con los hitos parciales y finales.
Integridad	Cada integrante del equipo garantiza la originalidad de su código y la transparencia del desarrollo colaborativo en el repositorio.
Responsabilidad y Puntualidad	Se fomenta el compromiso absoluto con las fechas límite, asumiendo que no existen prórrogas en el ámbito profesional.
Precisión y Pensamiento Lógico	El desarrollo exige la correcta modelación de problemas físicos complejos a través de abstracciones algorítmicas robustas.

El objetivo SMART que gobierna esta práctica de laboratorio define que, al finalizar un lapso estricto de veintiocho días, un equipo integrado por exactamente cuatro personas desarrollará un sistema web de simulación distribuida capaz de modelar el tránsito de paquetes de datos a través de una constelación satelital activa en tiempo real. El software deberá gestionar de forma segura múltiples puertos lógicos locales, validar datos estructurales mediante expresiones regulares avanzadas, estructurar la persistencia exclusivamente en memoria RAM a través de Tipos de Datos Abstractos diseñados manualmente y compilar reportes visuales dinámicos vectoriales SVG directamente desde la memoria RAM del servidor web. Este nivel de exigencia técnica evalúa y robustece de forma integral el conocimiento analítico del estudiante en la abstracción de memoria a bajo nivel, preparándolo para enfrentar con éxito los cursos de Estructura de Datos y Organización de Lenguajes y Compiladores I.

2. Marco Conceptual: Del Paradigma Teórico a la Práctica en C#

2.1 El Ecosistema .NET Core y la Gestión de Memoria Dinámica

La transición de lenguajes interpretados de tipado dinámico a lenguajes compilados de tipado estricto como C# introduce al estudiante en la gestión precisa del ciclo de vida de los objetos en la memoria RAM. En .NET, la memoria se divide de forma lógica en dos grandes estructuras: el Stack (Pila de ejecución) y el Heap (Montículo de asignación dinámica).

Las variables de tipo valor (como `int`, `double`, `struct`) se asignan directamente en el Stack, mientras que los objetos de tipo referencia (instancias de `class`) se alojan en el Heap, dejando únicamente un puntero físico de direccionamiento en el Stack. En este proyecto, cada nodo de las estructuras de datos manuales representa una asignación dinámica en el Heap. A diferencia de lenguajes como C o C++, donde el programador debe invocar manualmente sentencias de liberación física de memoria, el CLR (Common Language Runtime) de .NET implementa un sistema de gestión automática conocido como el Garbage Collector (Recolector de Basura). Sin embargo, el estudiante debe comprender el impacto de mantener referencias vivas (enlaces de punteros activos) en el grafo de objetos, ya que cualquier nodo que aún sea apuntado por un elemento raíz no podrá ser recolectado, pudiendo provocar fugas de memoria lógicas (Memory Leaks) si los enlaces de los nodos eliminados no se reconstruyen quirúrgicamente a nivel de código.

2.2 Eficiencia Algorítmica y Notaciones Asintóticas en las Estructuras de Datos

Para justificar la adopción de estructuras no lineales frente a estructuras lineales simples, el estudiante debe analizar el costo computacional de las operaciones fundamentales (búsqueda, inserción y eliminación). Este análisis se rige por la notación asintótica de la "O Grande" ($O()$), que describe el comportamiento del algoritmo en el peor escenario posible a medida que el tamaño de la entrada de datos n tiende al infinito.

- **Estructuras Lineales de Búsqueda Secuencial:** En una lista enlazada simple o doble, la búsqueda de un elemento requiere recorrer secuencialmente cada nodo de principio a fin, resultando en una complejidad de tiempo de $O(n)$.
- **Árboles Binarios de Búsqueda (ABB):** En un ABB con un factor de distribución equilibrado, cada bifurcación reduce a la mitad el espacio de búsqueda restante, logrando una complejidad promedio de $O(\log n)$. Sin embargo, ante inserciones ordenadas de forma secuencial, un ABB convencional degenera en una lista enlazada simple, elevando su complejidad temporal catastróficamente a $O(n)$.
- **Árboles Auto-Balanceados (AVL):** Para mitigar la degeneración lineal de los árboles, la estructura AVL implementa un factor de balanceo riguroso sobre la altura de sus subárboles. Mediante rotaciones

dinámicas (simples y dobles), garantiza que la diferencia de alturas nunca sea mayor a uno, manteniendo una complejidad inmutable de $O(\log n)$ tanto para la inserción como para la búsqueda y eliminación.

- **Matrices Dispersas Ortogonales:** En lugar de reservar un bloque contiguo bidimensional de tamaño $m \times n$ en memoria para almacenar coordenadas vacías, la matriz dispersa solo reserva espacio para elementos no nulos (nnz), enlazándolos a través de nodos cabecera de filas y columnas, optimizando significativamente la memoria espacial y el tiempo de barrido de la red.

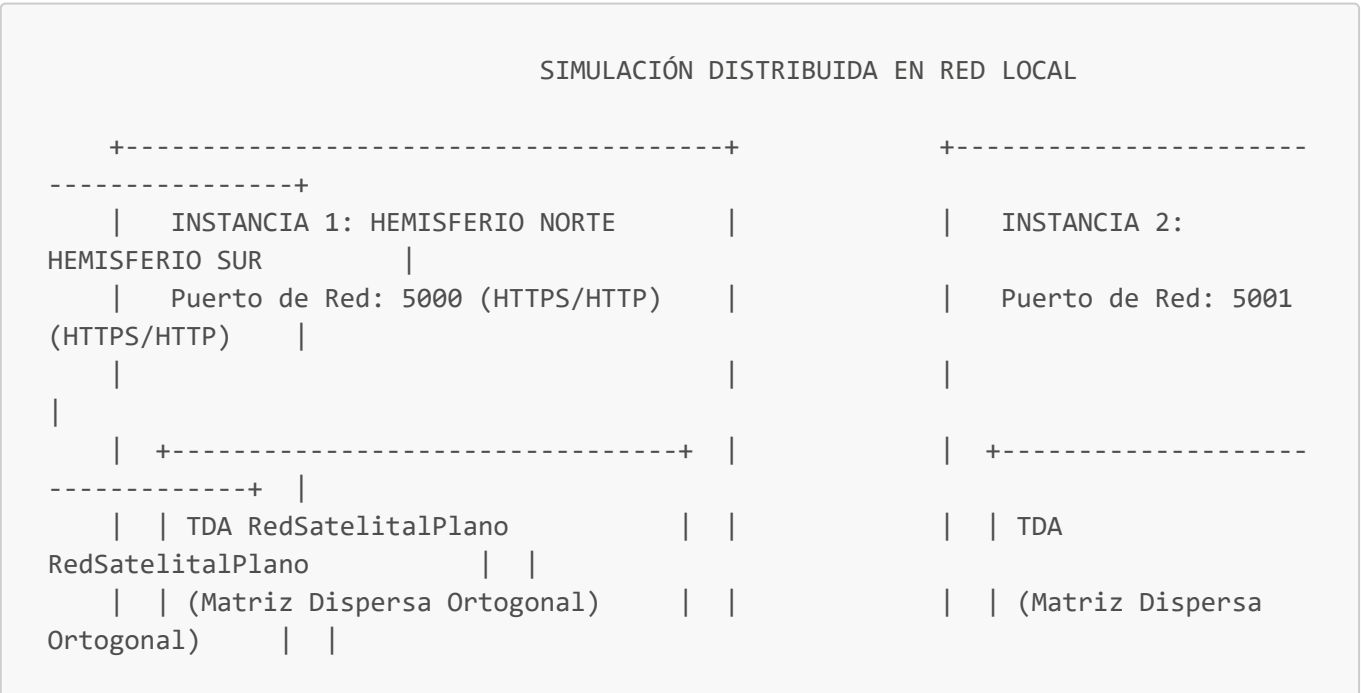
3. Enunciado Detallado del Simulador OrbitNet-NetCore

3.1 Descripción de la Red de Simulación Satelital

La simulación de redes de telecomunicaciones satelitales distribuidas representa un desafío de alta ingeniería debido al dinamismo del movimiento orbital y a la necesidad de garantizar la integridad del enrutamiento de datos a la superficie terrestre. El simulador desarrollado en esta práctica, denominado OrbitNet-NetCore, modelará la interacción entre satélites de órbita baja y media (divididos en constelaciones polares y ecuatoriales) y estaciones terrestres receptoras activas. El sistema representará el tránsito de paquetes de datos críticos que deben viajar de un satélite a otro utilizando enlaces lógicos para buscar la ruta de descarga óptima a la Tierra.

Debido a que el desarrollo de este simulador se ejecutará sobre el entorno estructurado y tipado de C# y la plataforma .NET, la persistencia se gestionará a través de un esquema estricto de clases autorreferenciadas que interactúan mediante punteros lógicos. El sistema operará bajo una arquitectura distribuida local, donde cada grupo de desarrollo deberá desplegar dos instancias simultáneas del servidor web ASP.NET Core MVC en la misma computadora. Una instancia representará la constelación del Hemisferio Norte (Puerto 5000) y la otra representará la del Hemisferio Sur (Puerto 5001).

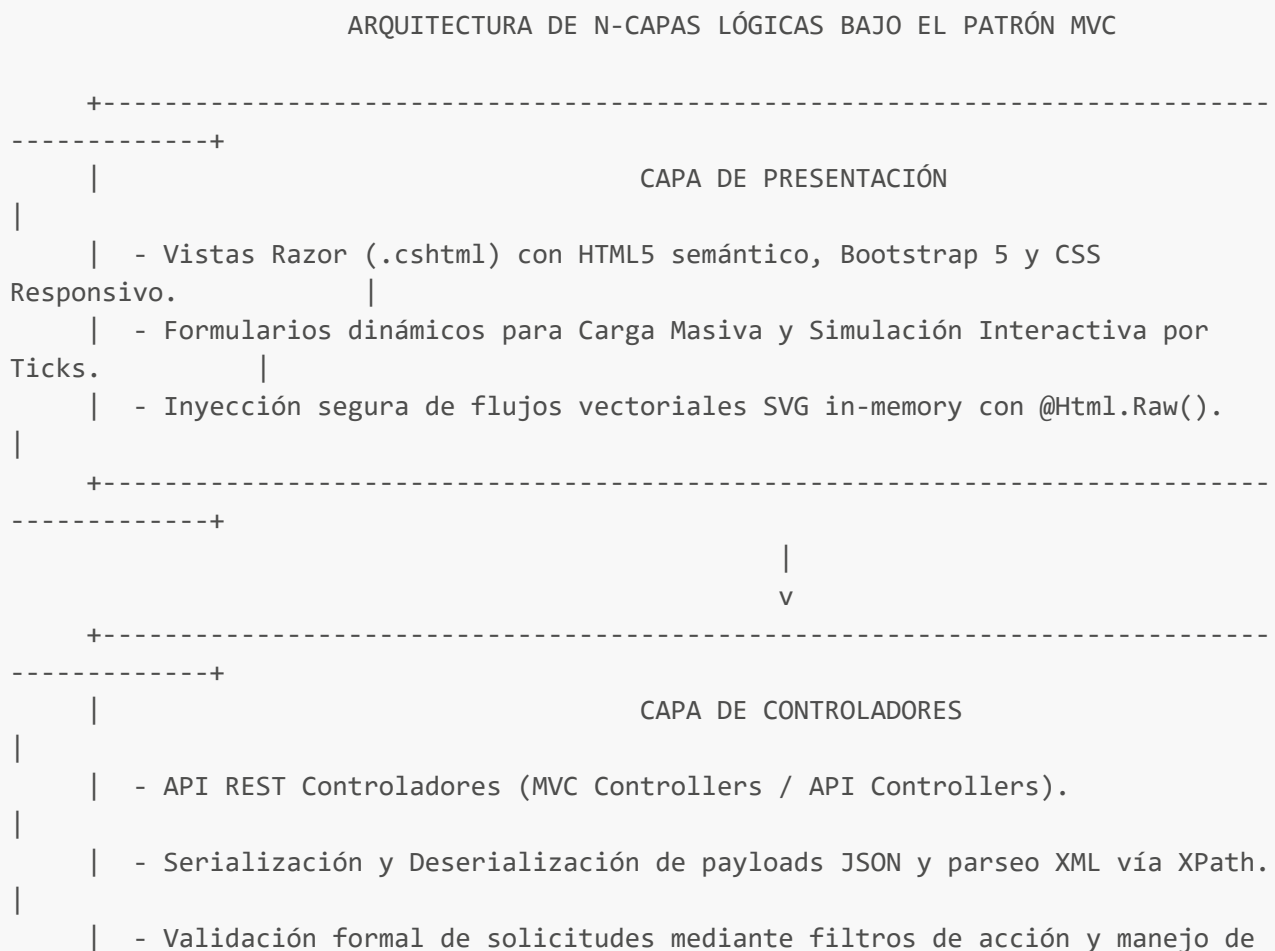
Cuando un paquete de datos ingresa a la constelación del Norte pero su destino físico corresponde a una estación terrestre configurada únicamente en la red de la constelación del Sur, el sistema deberá enrutar el paquete de forma interna a través de sus satélites locales y, al alcanzar el límite de la red, realizar un salto HTTP síncrono enviando una petición HTTP POST segura hacia la instancia hermana.

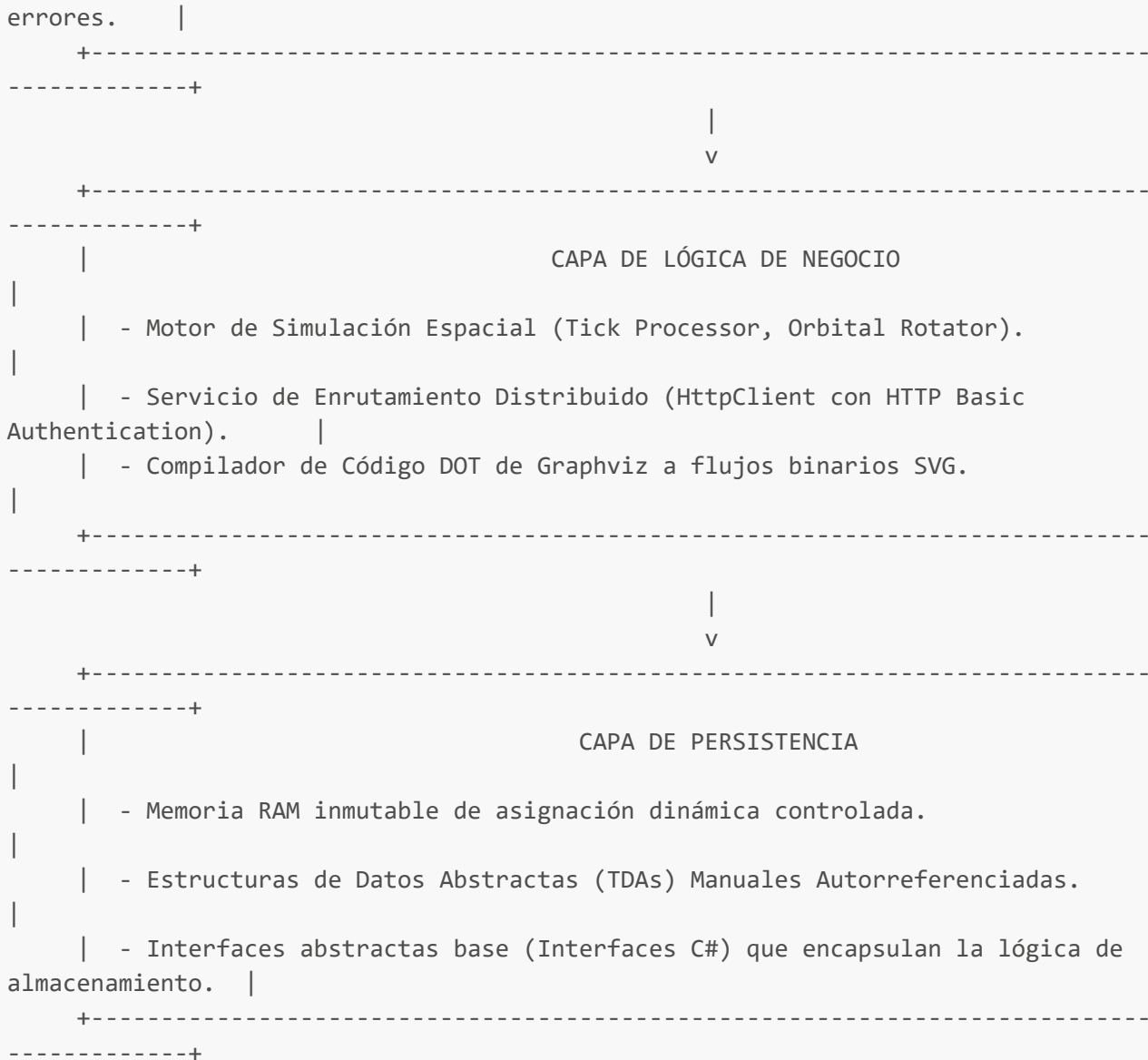




4. Arquitectura del Sistema y Patrón MVC en ASP.NET Core

Para cumplir con las competencias de arquitectura de software empresarial, el simulador se estructurará bajo el patrón Modelo-Vista-Controlador (MVC), implementando una arquitectura de capas lógicas acopladas de manera débil mediante interfaces de software.





4.1 Reglas Estrictas de Navegabilidad y Flujo Típico

- **Regla de Dependencia Unidireccional:** El flujo de llamadas lógicas se realizará estrictamente de arriba hacia abajo: Vista -> Controlador -> Capa de Servicio -> Capa de Persistencia (TDAs). Queda terminantemente prohibido que las clases de persistencia o nodos invoquen elementos controladores o rendericen marcado de presentación de manera directa.
- **Flujo de Ejecución de una Operación (Secuencia Típica):**
 1. El usuario pulsa el botón "Avanzar Simulación" en la Vista.
 2. La Vista envía una petición HTTP POST asíncrona al Controlador.
 3. El Controlador recibe la petición, valida la sesión del usuario y delega la ejecución al Motor de Simulación (Capa de Servicio).
 4. El Motor de Simulación invoca los métodos de rotación cíclica en la matriz dispersa y despacha el buffer de mensajes de máxima prioridad del TDA de almacenamiento.
 5. Los TDAs modifican las referencias físicas de sus nodos en memoria RAM de forma atómica.
 6. El servicio de Graphviz compila el nuevo estado de la memoria en caliente como un flujo SVG.
 7. El Controlador recibe el SVG, lo inyecta en el ViewModel y retorna la respuesta de refresco dinámico a la Vista.

5. Especificación Algorítmica de los TDAs en RAM (Strict C#)

Para garantizar la correcta evaluación de la destreza lógica del estudiante, no se podrá hacer uso de clases o interfaces pertenecientes a `System.Collections` o `System.Collections.Generic`. Todas las estructuras deben codificarse utilizando punteros de tipo objeto referenciados manualmente en C#.

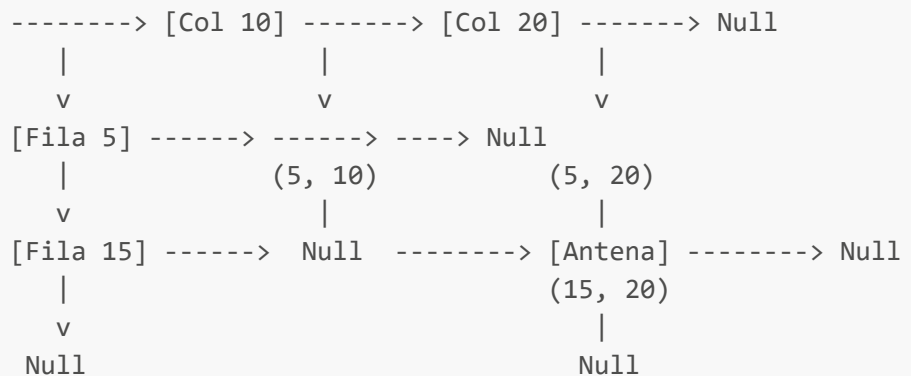
A continuación, se detalla la especificación algorítmica y la interfaz de programación obligatoria para cada TDA:

```
// Firma base abstracta que debe implementar todo TDA para control de auditoría
public interface IAbstractCollection
{
    int Count { get; }
    void Clear();
    bool IsEmpty { get; }
}
```

5.1 TDA RedSatelitalPlano: Matriz Dispersa Ortogonal Bidireccional

Modela la topología espacial tridimensional proyectada en un plano cartesiano de latitudes y longitudes enteras. La matriz contiene nodos cabecera para controlar los límites lógicos de las coordenadas (\$X\$ e \$Y\$) y nodos de datos enlazados de forma ortogonal.

REPRESENTACIÓN LÓGICA DE LA MATRIZ DISPERSA ORTOGONAL



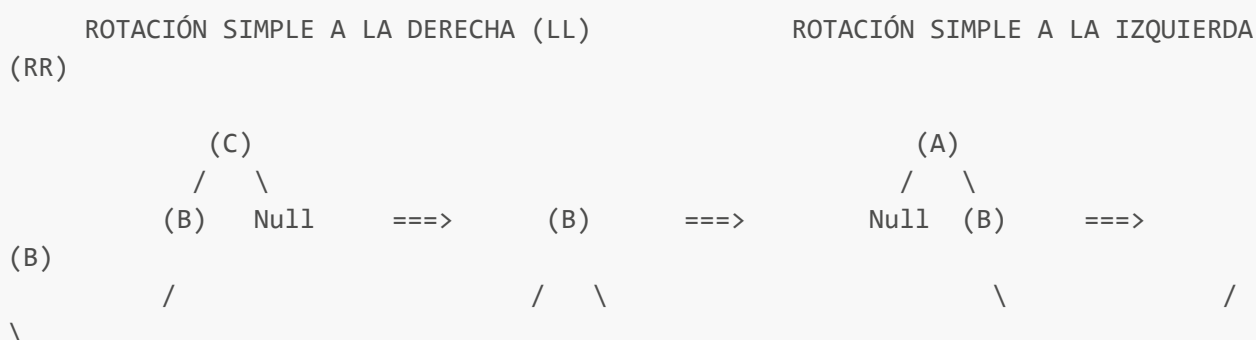
- **Estructura del Nodo Cabecera (HeaderNode):**
 - `int Index`: Coordenada numérica (fila o columna).
 - `HeaderNode Next`: Apuntador al siguiente cabecera del eje.
 - `MatrixNode Access`: Apuntador directo al primer nodo de datos asociado a esta cabecera.
- **Estructura del Nodo de Datos (MatrixNode):**
 - `int Row, int Col`: Coordenadas lógicas dentro de la matriz.
 - `string Id`: Identificador único validado.
 - `string IpAddress`: Dirección IPv4.
 - `MatrixNode Up, MatrixNode Down, MatrixNode Left, MatrixNode Right`: Punteros de enlazado ortogonal bidireccional.

- `TDA_BufferSatelital Buffer`: Instancia única del buffer interno de mensajes para este satélite.
- **Operaciones Matemáticas y de Estructura Obligatorias:**
 - `void Insert(int row, int col, string id, string ip)`: Crea las cabeceras correspondientes si no existen e inserta de forma ordenada de menor a mayor por coordenada el nodo de datos. Debe validar que no exista colisión de coordenadas.
 - `void Delete(int row, int col)`: Elimina físicamente el nodo del plano orbital y reconecta quirúrgicamente los cuatro punteros ortogonales adyacentes para prevenir discontinuidades en la malla de datos.
 - `MatrixNode Search(int row, int col)`: Retorna la instancia de datos ubicada en la posición especificada en tiempo $O(r + c)$.

5.2 TDA RegistroSatelites: Árbol AVL Auto-Balanceado

Funciona como el catálogo global e inmutable de satélites activos e históricos del simulador. Al ser un árbol auto-balanceado, optimiza los tiempos de búsqueda masiva a un máximo constante de $O(\log n)$ comparaciones.

- **Estructura del Nodo AVL (AvlNode):**
 - **string SatelliteId:** Llave de búsqueda única.
 - **string Name:** Nombre identificador.
 - **double Frequency:** Frecuencia de operación lógica.
 - **int Height:** Altura del nodo en el subárbol para cálculos de balanceo.
 - **AvlNode LeftChild, AvlNode RightChild:** Punteros lógicos a los hijos.
- **Balanceo Dinámico por Altura (Rotaciones):** El factor de equilibrio de un nodo se calcula mediante la fórmula matemática: $BF = H_{\text{der}} - H_{\text{izq}}$ Donde H_{der} representa la altura del subárbol derecho y H_{izq} la altura del subárbol izquierdo. Un nodo se considera desbalanceado si su factor de equilibrio absoluto $|BF| \geq 2$. Para corregir este estado, se deben codificar de forma manual los cuatro tipos de rotaciones fundamentales:
 - **Rotación Simple a la Izquierda (RR):** Aplicada cuando la inserción ocurre en el hijo derecho del subárbol derecho desbalanceado.
 - **Rotación Simple a la Derecha (LL):** Aplicada cuando la inserción ocurre en el hijo izquierdo del subárbol izquierdo desbalanceado.
 - **Rotación Doble Izquierda-Derecha (LR):** Consiste en una rotación simple izquierda sobre el hijo izquierdo, seguida de una rotación simple derecha sobre la raíz del desbalance.
 - **Rotación Doble Derecha-Izquierda (RL):** Consiste en una rotación simple derecha sobre el hijo derecho, seguida de una rotación simple izquierda sobre la raíz del desbalance.



(C)	(A)	(A)	(C)	(C)	(A)
-----	-----	-----	-----	-----	-----

5.3 TDA BufferMensajes: Árbol Binario de Búsqueda (ABB) como Cola de Prioridad

Cada satélite de la matriz dispersa posee una esclusa o buffer de procesamiento no lineal para almacenar los mensajes recibidos. Este buffer implementa un Árbol Binario de Búsqueda ordenado según el nivel de prioridad del paquete de datos.

- **Estructura del Nodo ABB (AbbNode):**

- `string HexCode`: Código hexadecimal único del paquete.
- `string EmisorId`: Satélite de origen.
- `string DestIp`: Dirección IP del nodo terrestre de destino.
- `int Priority`: Nivel de prioridad (valores enteros del 1 al 5).
- `string Content`: Cuerpo de texto del mensaje.
- `AbbNode Left`, `AbbNode Right`: Enlaces lógicos.

- **Operaciones de Cola de Prioridad sobre ABB:**

- `void Enqueue(AbbNode packet)`: Inserta el paquete de datos en el árbol. La ordenación principal se rige por la prioridad: los valores de prioridad mayor (\$5\$ - Alerta Crítica) se ubican en la extrema derecha del árbol para ser despachados primero, mientras que las de menor valor se ubican a la izquierda.
- `AbbNode Dequeue()`: Extrae y retorna el mensaje de máxima prioridad in-memory (el nodo ubicado más a la derecha en el árbol). Se debe reestructurar el árbol reconectando los hijos del nodo extraído aplicando la lógica de eliminación estándar de ABB.
- `void TraverseInOrder()`: Recorrido recursivo que retorna los mensajes ordenados de forma prioritaria para auditoría.

5.4 TDA LogAuditoria: Lista Enlazada Simple

Registra de forma cronológica, lineal e inmutable cada evento, advertencia, error de validación e intrusión de red detectado por el motor del simulador.

- **Estructura del Nodo Log (LogNode):**

- `DateTime Timestamp`: Fecha y hora exacta de la transacción.
- `string Severity`: Gravedad (INFO, ALERT, ERROR).
- `string Message`: Descripción detallada del suceso.
- `LogNode Next`: Puntero al siguiente evento.

- **Operaciones del Log:**

- `void WriteEvent(string severity, string msg)`: Inserta un nuevo log en tiempo $O(1)$ al final de la lista utilizando un puntero de cola (Tail pointer).
- `string SearchLogRegex(string pattern)`: Filtra y retorna los logs cuya descripción coincida con una expresión regular dada.

6. Motor de Ingesta, Carga Masiva y Validaciones RegEx

El backend del sistema procesará de forma síncrona el archivo estructurado provisto por el usuario a través del portal de carga masiva, analizando la topología y poblando las estructuras de datos dinámicas manuales.

6.1 Parseo XML Avanzado mediante XPath

Para evitar la iteración recursiva desordenada de elementos, el backend implementará de forma obligatoria la clase nativa `XmlDocument` de .NET en conjunto con un `XmlNamespaceManager` para evaluar consultas de selección XPath estructuradas directas:

```
// Ejemplo de Carga Masiva e Ingesta XPath Segura en ASP.NET Core
public void ProcessXmlConfiguration(string xmlContent, MatrixDispDispersa matriz,
AvlTree catalogo)
{
    XmlDocument doc = new XmlDocument();

    // Configuración OWASP para mitigar vulnerabilidades de Inyección de Entidades
    Externas (XXE)
    doc.XmlResolver = null;

    doc.LoadXml(xmlContent);

    // Extracción de Satélites de órbita ecuatorial
    XmlNodeList satEcuatoriales =
doc.SelectNodes("//constelaciones_ecuatoriales/satelite"); [1]
    foreach (XmlNode sat in satEcuatoriales)
    {
        string id = sat.Attributes["id"]?.Value;
        string nombre = sat.SelectSingleNode("nombre")?.InnerText;
        string ip = sat.SelectSingleNode("enlace_ip")?.InnerText;

        if (ValidateTelemetryData(id, ip, "14.0000,-90.0000")) // Coordinada por
defecto para Ecuador
        {
            matriz.Insert(0, int.Parse(id.Substring(8)), id, ip);
        }
    }
}
```

6.2 Validaciones Sintácticas Rigurosas mediante RegEx

Antes de proceder con la instanciación de cualquier clase nodo o su inserción en la memoria RAM, el módulo de ingesta debe ejecutar una validación formal de los datos de entrada empleando la clase `System.Text.RegularExpressions.Regex`. Las reglas sintácticas de validación se exponen en la siguiente tabla de expresiones regulares estándar:

Atributo Validado	Expresión Regular de Validación	Ejemplo Válido	Comportamiento ante Fallo de Validación
-------------------	---------------------------------	----------------	---

Atributo Validado	Expresión Regular de Validación	Ejemplo Válido	Comportamiento ante Fallo de Validación
ID de Satélite	<code>^SAT-(ECU\ POL)-\d{4}\$</code>	<code>SAT-ECU-0012</code>	Rechazo completo de la carga masiva, generación de alerta en la interfaz web y registro de evento tipo ERROR en el TDA LogAuditoria.
Dirección IPv4	<code>^(?:25[0-5]\ 2[0-4][0-9]\ ?[0-9][0-9]?)\.(?:25[0-5]\ 2[0-4][0-9]\ ?[0-9][0-9]?)\$</code>	<code>10.0.0.50</code>	Descarta la inserción del nodo correspondiente, aborta la transacción y emite alerta detallada en la UI.
Coordenadas	<code>^-?\d{1,2}\.\d{4,6},-?\d{1,3}\.\d{4,6}\$</code>	<code>14.5891,-90.5514</code>	Anulación automática de la ingesta, liberación de los nodos parcialmente instanciados y escritura de alerta.

El motor de carga masiva operará bajo un esquema transaccional atómico. Si un elemento del archivo XML de configuración contiene campos corruptos o inconsistencias de formato frente a las expresiones regulares declaradas, la carga completa debe cancelarse inmediatamente en memoria. El sistema no debe dejar estados inconsistentes o estructuras a medio poblar; se restaurará el estado previo, se mostrará en pantalla el error exacto indicando la línea y etiqueta inválida, y se registrará un log de severidad ERROR detallando la anomalía.

7. Protocolo REST Multi-Puerto y Comunicación Inter-Satélite

La simulación distribuida e interconectada de OrbitNet-NetCore exige el despliegue de dos procesos de servidor ASP.NET Core MVC concurrentes en un mismo equipo local de desarrollo, asignando puertos diferentes para cada constelación.





7.1 Comunicación REST mediante HttpClient

Para comunicar ambas instancias de forma distribuida en tiempo real, se inyectará la clase `IHttpClientFactory` a través del contenedor de dependencias nativo de ASP.NET Core. Al detectarse que el destino final de un paquete tiene una dirección IP asignada a una antena de la constelación hermana, se disparará una petición POST síncrona.

7.2 Capa de Seguridad de Red: HTTP Basic Authentication

Todas las comunicaciones REST entre puertos se autenticarán obligatoriamente a través del estándar industrial HTTP Basic Authentication.

- **Construcción de Cabeceras:** La instancia emisora tomará el usuario lúdico `orbitnet_admin` y la contraseña de seguridad `USAC_ECYS_2026`, formateándolas como una cadena concatenada `orbitnet_admin:USAC_ECYS_2026`.
- **Codificación Base64:** Dicha cadena se codifica en Base64 resultando en `b3JiaXRuZXRfYWRTaW46VVNBQ19FQ1lTXzIwMjY=`.
- **Inyección de Cabecera:** Se añade a la petición HTTP el encabezado: `Authorization: Basic b3JiaXRuZXRfYWRTaW46VVNBQ19FQ1lTXzIwMjY=`. El servidor receptor interceptará este valor, decodificará las credenciales y rechazará cualquier conexión no autorizada con un código HTTP 401 Unauthorized.

8. Motor de Visualización en Memoria con Graphviz (C# Process & SVG Redirection)

La comprobación irrefutable del manejo correcto de las referencias de memoria, enlaces físicos de punteros lógicos y operaciones dinámicas sin el uso de persistencia de bases de datos tradicionales constituye el componente técnico de mayor relevancia en la calificación del proyecto. El portal web de cada instancia incorporará un módulo interactivo para mostrar el estado en tiempo real del sistema.

8.1 Arquitectura de Renderizado Sin Persistencia Física en Disco

Para evitar problemas de concurrencia y bloqueos lógicos del sistema operativo, queda estrictamente prohibido que el servidor web escriba o almacene archivos de imagen temporales (.png, .jpg) o scripts de diagramación plano (.dot) en el disco duro local. La compilación de código DOT del motor de Graphviz se ejecutará en caliente, procesando la información directamente en memoria RAM y redireccionando la salida estándar del subprocesso del sistema (dot.exe) como un flujo de bytes SVG en tiempo real.

8.2 Evitando Bloqueos Lógicos (Deadlocks) de Procesos en C#

Un error recurrente al utilizar redirección de flujos estándar (RedirectStandardOutput y RedirectStandardInput) con subprocessos del sistema en .NET es el bloqueo lógico o deadlock. Este fenómeno ocurre porque los buffers de tubería del sistema operativo tienen un tamaño de almacenamiento limitado. Si el subprocesso hijo (dot.exe) escribe datos SVG que superan la capacidad del buffer de salida mientras el proceso padre de C# espera a que el proceso finalice llamando a dot.WaitForExit(), ambos procesos se bloquearán indefinidamente.

Para solventar esto de forma segura en C#, se debe implementar una lectura asíncrona o escribir y cerrar el canal de entrada (StandardInput) antes de leer completamente el flujo de salida.

```
public static string CompileDotToSvg(string dotSourceCode)
{
    using (Process process = new Process())
    {
        process.StartInfo.FileName = "dot";
        process.StartInfo.Arguments = "-Tsvg";
        process.StartInfo.UseShellExecute = false;
        process.StartInfo.RedirectStandardInput = true;
        process.StartInfo.RedirectStandardOutput = true;
        process.StartInfo.RedirectStandardError = true;
        process.StartInfo.CreateNoWindow = true;

        process.Start();

        // Escritura asíncrona o directa en el stream de entrada
        using (StreamWriter writer = process.StandardInput)
        {
            writer.Write(dotSourceCode);
            writer.Flush();
        } // Cerrar el StandardInput le indica a dot.exe que la entrada ha
        finalizado [12]

        // Leer la salida estándar antes de invocar WaitForExit previene el
        deadlock
        string svgOutput = process.StandardOutput.ReadToEnd();
        string errorOutput = process.StandardError.ReadToEnd();

        process.WaitForExit();

        if (process.ExitCode != 0)
        {
            throw new Exception($"Error de compilación Graphviz: {errorOutput}");
        }
    }
}
```

```

        return svgOutput;
    }
}

```

Este flujo vectorial SVG se inyectará de manera directa en las vistas de Razor de ASP.NET Core utilizando el filtro que elude el escape automático de caracteres HTML de seguridad: `@Html.Raw(Model.SvgData)`. El módulo interactivo de visualización generará tres reportes avanzados obligatorios:

8.3 Reporte 1: Estado Físico de Memoria de los TDAs (Memory Layout Map)

Representa un mapa visual fidedigno que demuestra la disposición en memoria física de los punteros lógicos en tiempo real para las estructuras dinámicas manuales construidas.

- **Requerimiento Técnico de DOT:** Todos los nodos de la estructura seleccionada deben diagramarse con formas de tipo registro (`shape=record`). Cada registro se subdividirá en campos explícitos para ilustrar la dirección física o puntero anterior (`prev`), el valor del dato encapsulado (`data`) y la dirección física o puntero siguiente (`next`).
- **Conexión de Aristas:** Las flechas que enlazan los nodos deben nacer directamente del puerto interno central del campo correspondiente (`:c`) configurando la directiva `tailclip=false` en el generador. Esto evitará colisiones desordenadas en las fronteras externas de las celdas de datos. En las estructuras que implementan comportamientos circulares, los punteros de retorno no deben cruzar diagonalmente el diagrama; se exige desactivar restricciones lógicas de jerarquía (`constraint=false`) para forzar que el graficador dibuje curvas exteriores limpias fuera de la geometría lineal principal.

```

/* Código DOT Oficial Recomendado para el TDA RedSatelitalPlano en RAM */
digraph MemoryLayout {
    rankdir=LR;
    node [shape=record, fontname="Consolas", fontsize=10];
    edge [tailclip=false, arrowsize=0.8];

    node_sat0;
    node_sat1;

    node_sat0:next:c -> node_sat1:data;
    node_sat1:prev:c -> node_sat0:data [color="#E74C3C", arrowhead=vee];
}

```

8.4 Reporte 2: Trazabilidad de Ruta de Retransmisión (Relay Route Tracer)

Al momento de completarse el enrutamiento y despacho de un paquete de datos a través de la constelación activa, el sistema generará de forma interactiva un grafo dirigido de la red de transporte para rastrear el histórico de saltos físicos.

- **Características de Pintura Dinámica:** Los satélites y antenas terrestres por los cuales transitó el paquete de manera exitosa se colorearán con un relleno verde brillante (`fillcolor="#2ECC71", style="filled"`). Aquellas estaciones o satélites que se encontraban inactivos, con colapso de tráfico o fuera de cobertura se representarán en gris con contornos discontinuos (`style="dashed, filled"`,

`fillcolor="#E74C3C"`). Las aristas que describan la ruta óptima de tránsito se renderizarán con un grosor destacado (`penwidth=3.0`, `color="#27AE60"`).

8.5 Reporte 3: Matriz Unificada de Capacidad y Estado del Buffer Satelital

Presenta un mapa bidimensional consolidado de la densidad del tráfico y el porcentaje de ocupación de las colas de prioridad de todos los satélites activos de la constelación.

- **Requerimiento Técnico de DOT:** Para evitar el desbordamiento de nodos individuales, este reporte utilizará etiquetas estructuradas basadas en sintaxis de tablas HTML soportadas de forma nativa por el procesador de Graphviz (`label=<<table border="0" cellpadding="1">...</table>>`).
- **Estructura de Información:** Cada celda renderizada en el diagrama representará un satélite de la matriz ortogonal dispersa, exponiendo su nombre identificador, su capacidad física total y un medidor gráfico de ocupación coloreado dinámicamente que servirá como barra de progreso visual de saturación, listando al final los identificadores de los mensajes que se encuentran en su cola de prioridad.

9. Guía Detallada de los Endpoints de la API

La API REST del simulador operará de manera distribuida, utilizando los siguientes esquemas de comunicación JSON para controlar los estados del sistema:

9.1 Endpoint 1: Ingesta de Configuración XML

- **Método HTTP:** `POST`
- **Endpoint:** `/api/v1/space/config`
- **Payload de entrada (JSON):**

```
{
  "xml_data": "<?xml version=\"1.0\"?><orbitnet><constelaciones_ecuatoriales>
<satelite id=\"SAT-ECU-0001\"><nombre>Starlink-N1</nombre>
<enlace_ip>127.0.0.1</enlace_ip></satelite></constelaciones_ecuatoriales>
</orbitnet>"
}
```

- **Respuesta de Éxito (200 OK):**

```
{
  "status": "Success",
  "message": "Configuración cargada exitosamente en RAM. Nodos procesados: 4.",
  "timestamp": "2026-06-01T17:56:00Z"
}
```

- **Respuesta de Error (400 Bad Request):**

```
{
  "status": "Error",
  "error_code": "XML_SCHEMA_VIOLATION",
  "details": "El ID de Satelite 'SAT-ECU-99' no cumple con el formato RegEx exigido."
}
```

9.2 Endpoint 2: Transferencia y Enrutamiento Inter-Satelital (Basic Auth requerido)

- **Método HTTP:** POST
- **Endpoint:** /api/v1/space/relay
- **Payload de entrada (JSON):**

```
{
  "codigo_hex": "A19F",
  "emisor_id": "SAT-ECU-0012",
  "destino_ip": "10.0.0.50",
  "prioridad": 5,
  "contenido": "Alerta de tsunami detectada en boya de superficie."
}
```

- **Respuesta de Éxito (201 Created):**

```
{
  "status": "Routed",
  "message": "Mensaje insertado con éxito en el buffer de prioridad del satélite receptor SAT-POL-1001.",
  "queue_occupancy_percentage": 40.0
}
```

- **Respuesta de Error de Credenciales (401 Unauthorized):**

```
{
  "status": "Unauthorized",
  "details": "Acceso restringido. Cabecera HTTP Basic Auth inválida o ausente."
}
```

9.3 Endpoint 3: Avance de Simulación (Ticks)

- **Método HTTP:** POST
- **Endpoint:** /api/v1/space/simulation/step
- **Payload de entrada (JSON):**

```
{
  "ticks": 1
}
```

- **Respuesta de Éxito (200 OK):**

```
{
  "status": "Simulated",
  "current_tick": 45,
  "events_processed": 3,
  "details": "Órbitas rotadas exitosamente. Se ejecutaron 2 saltos lógicos."
}
```

10. Plan de Calidad, Pruebas Unitarias y Seguridad OWASP

Para garantizar el cumplimiento de los estándares profesionales de desarrollo de software, el proyecto integrará de forma obligatoria un enfoque sistemático de calidad de software (Quality Assurance) y seguridad informática.

10.1 Estrategia de Pruebas Automatizadas (xUnit/NUnit)

Cada grupo de desarrollo deberá programar un proyecto de pruebas unitarias desacoplado de la solución principal, utilizando el framework xUnit o NUnit de .NET. Las pruebas automatizadas deben cubrir los siguientes aspectos críticos:

- **Pruebas de Inserción y Balanceo AVL:** Validar que, al insertar nodos de forma desordenada en el árbol AVL de satélites, la altura del árbol permanezca balanceada con una diferencia no mayor a uno ($|FE| \leq 1$) y se ejecuten correctamente las rotaciones simples y dobles.
- **Pruebas de Búsqueda y Enlace de Matriz Dispersa:** Validar la exactitud de los punteros internos tras eliminar un nodo ubicado en medio de la matriz ortogonal.
- **Pruebas de Encolamiento por Prioridades en ABB:** Asegurar que, al despachar los elementos del buffer, los mensajes con prioridad alta (\$5\$) siempre sean retornados antes de las prioridades inferiores (\$1\$), evaluando condiciones límite.

10.2 Pruebas de Integración y Simulaciones Locales de Red

Se debe automatizar un set de pruebas de integración para simular el comportamiento concurrente de los dos servidores lógicos. Mediante scripts de pruebas se enviarán paquetes de datos al puerto \$5000\$ y se verificará de forma automatizada mediante llamadas asíncronas que el mensaje haya completado el viaje por el canal de comunicación del puerto \$5001\$.

10.3 Seguridad en Aplicaciones Web (OWASP Top 10)

El backend en C# debe incorporar de forma activa defensas contra vulnerabilidades comunes de nivel empresarial, alineadas con los estándares OWASP:

- **Mitigación de Vulnerabilidades XXE (Inyección de Entidades Externas):** El procesador `XmlDocument` utilizado para la ingesta de archivos masivos XML se configurará explícitamente para deshabilitar la resolución de entidades externas y la validación de archivos de definición de esquemas DTD de origen desconocido, previniendo inyecciones de datos y ataques de denegación de servicio en el servidor. El programador configurará `doc.XmlResolver = null` de manera estricta.
 - **Prevención de Inyecciones Cross-Site Scripting (XSS):** Debido al uso obligatorio de inyección en bruto `@Html.Raw()` para incrustar el SVG renderizado en memoria dentro de las vistas de Razor, el backend debe sanitizar el código de marcado de Graphviz de forma previa a su renderizado. De esta forma se neutralizará la posibilidad de inyectar scripts maliciosos en los atributos de etiquetas HTML del gráfico.
-

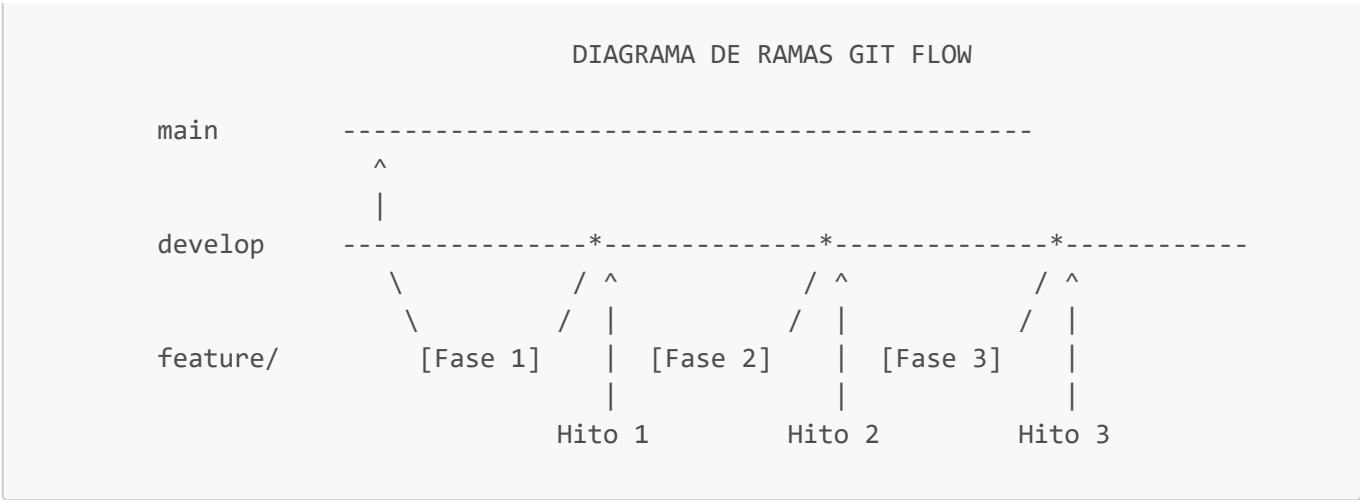
11. Entregables y Estructura del Repositorio

Los grupos de desarrollo empaquetarán y subirán a la plataforma virtual académica oficial UEDI de la universidad, antes de la fecha límite (29 de junio de 2026 a las 23:59 horas), los siguientes entregables estructurados:

- **Código Fuente Completo:** Estructurado de forma modular en una solución de .NET desacoplada en proyectos diferenciados de biblioteca de clases para los TDAs manuales, el motor de la API REST del backend, las vistas de la aplicación web y el módulo de pruebas automatizadas. No se admitirá código desorganizado o unificado en un solo archivo de compilación.
 - **Repositorio Privado de GitHub:** Configurado estrictamente bajo el formato de nomenclatura: `IPC2_Proyecto_2026_Grupo[NumeroGrupo]`. Se debe registrar de forma obligatoria la incorporación del auxiliar de cátedra asignado (`@FJVicVe103`) como colaborador directo del repositorio para permitir la auditoría de código.
 - **Manual Técnico (MANUAL_TECNICO.md):** Documento formal redactado en formato Markdown que describa detalladamente la arquitectura lógica adoptada para la construcción de los cuatro TDAs manuales, diagramas UML de clases con las firmas de métodos principales, documentación de los endpoints expuestos por la API RESTful (cuerpo JSON de peticiones y respuestas con códigos de estado HTTP) y las estrategias de seguridad de datos implementadas.
 - **Manual de Usuario (MANUAL_USUARIO.md):** Guía didáctica en formato Markdown enriquecida con capturas de pantalla que ilustren detalladamente el funcionamiento de la interfaz de carga masiva XML, la administración de la comunicación distribuida local inter-puertos, el disparo y avance secuencial de los ticks de la simulación y la navegación interactiva de los tres reportes de diagnóstico SVG.
 - **Diagrama de Flujo del Simulador:** Esquematización gráfica técnica que describa el flujo de control que sigue un paquete de datos desde su ingreso al sistema web, su tránsito por la matriz dispersa y su posterior enrutamiento REST cross-port entre las instancias de red locales.
-

12. Cronograma de Hitos de Control y Git Flow

Para incentivar la constancia en el desarrollo técnico, evitar la acumulación de actividades en las fechas límite y garantizar que la práctica de programación constituya un proceso colaborativo real, el ciclo de desarrollo se divide en tres fases obligatorias con hitos de control funcionales de evaluación individual:



Fase Evaluativa	Plazo de Ejecución	Ponderación	Hito Técnico y Objetivos Funcionales de Desarrollo
Fase 1	Del 1 al 9 de junio de 2026	25%	Configuración de Entorno, Ingesta de Datos y Validaciones RegEx: Estructuración de la solución de .NET e inicialización del repositorio GitHub aplicando la metodología estricta de ramificación Git Flow (ramas estables <code>main</code> , rama de integración continua <code>develop</code> , y ramas temporales de características <code>feature/</code>). Programación del motor de ingesta masiva XML empleando XPath con la clase <code>XmlDocument</code> y validación robusta de datos con expresiones regulares (<code>Regex.IsMatch()</code>). Hito: Carga masiva exitosa y validación atómica de archivos en la interfaz web de la aplicación.
Fase 2	Del 10 al 19 de junio de 2026	35%	Desarrollo del Core de TDAs Manuales y Visualizaciones Graphviz: Programación en C# de los cuatro Tipos de Datos Abstractos manuales aplicando encapsulamiento técnico y polimorfismo estructurado. Desarrollo del compilador dinámico de Graphviz en caliente que procese las instrucciones de diseño DOT directamente en la memoria RAM del servidor web sin escrituras físicas en disco, retornando flujos de datos vectoriales SVG de visualización interactiva. Hito: Compilación e inyección dinámica del plano de punteros y buffers en las vistas Razor.

Fase Evaluativa	Plazo de Ejecución	Ponderación	Hito Técnico y Objetivos Funcionales de Desarrollo
Fase 3	Del 20 al 29 de junio de 2026	40%	Despliegue de API REST, Comunicación Multi-Puerto y Entrega Final: Estructuración de los endpoints de la API REST utilizando controladores MVC en ASP.NET Core. Configuración para habilitar el levantamiento simultáneo de los servidores locales en los puertos lógicos 5000 y 5001. Programación de la comunicación cross-port asíncrona autenticada mediante HTTP Basic Authentication. Implementación del banco de pruebas unitarias y de integración empleando xUnit o NUnit. Hito: Entrega final del simulador completamente integrado y defensa individual presencial.

Cada integrante del equipo debe registrar de manera obligatoria un mínimo de tres commits significativos por semana dentro del repositorio de GitHub. El cumplimiento de esta métrica es auditado individualmente; no se admitirán repositorios con commits masivos realizados de última hora o concentrados en un único desarrollador, aplicando penalizaciones severas individuales en caso de detectarse inactividad.

13. Políticas de Evaluación y Código de Ética (Tolerancia Cero)

La Escuela de Ciencias y Sistemas de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala rige sus actividades de aprendizaje bajo políticas estrictas de honestidad académica, modelando la conducta profesional de los estudiantes:

- **Tolerancia Cero al Plagio y Copias de Código:** Todos los proyectos serán sometidos de forma automatizada a revisiones cruzadas utilizando herramientas avanzadas de detección de similitudes estructurales de código de programación (tales como MOSS de la Universidad de Stanford o analizadores de sintaxis propios de la cátedra). Si se determina que dos o más grupos comparten lógica de clases o han copiado componentes funcionales de repositorios públicos o proyectos de semestres anteriores, se asignará de manera automática una nota de cero puntos (0) a la totalidad de los estudiantes involucrados. El caso se derivará formalmente a la dirección académica de la Escuela para las respectivas sanciones disciplinarias de suspensión.
- **Prohibición de Código Generado por Inteligencia Artificial sin Dominio:** El propósito de la práctica es el desarrollo de habilidades de modelado y abstracción lógica de memoria a bajo nivel. El uso de asistentes de IA generativa (como ChatGPT, Claude o Copilot) para copiar bloques de código directamente sin que el estudiante demuestre capacidad para modificar, depurar y defender la lógica de las estructuras frente al auxiliar de cátedra, resultará en la anulación automática del proyecto (0 puntos).
- **Requisitos de Asistencia and Aprobación:** Para validar la zona del laboratorio, el estudiante debe cumplir con una asistencia mínima obligatoria del 80% a las sesiones programadas. La nota de aprobación del componente de laboratorio de la práctica es de 61 puntos netos sobre 100. Aprobar el laboratorio es un requisito indispensable y obligatorio para tener derecho a presentarse a la evaluación teórica final del curso de Introducción a la Programación y Computación II.

14. Rúbrica Detallada de Calificación (100 Puntos)

La evaluación cuantitativa del Proyecto Único se desglosa en criterios de desarrollo técnico y de funcionamiento operativo del sistema para garantizar una calificación transparente:

14.1 Sección de Desarrollo Técnico (Habilidades) - 40 Puntos

Criterio Evaluado	Ponderación	Aspectos Técnicos de Calificación en C# y .NET Core
Calidad Estructural y POO	15 Puntos	Modularización correcta en proyectos desacoplados. Uso estricto de encapsulamiento mediante propiedades automáticas, abstracción de clases nodo con herencia y control de dependencias con NuGet.
Control de Versiones y Git Flow	10 Puntos	Historial de commits limpio que demuestre la adopción exacta de la metodología Git Flow (ramas <code>develop</code> , <code>feature</code> , <code>main</code>). Evidencia de commits semanales distribuidos equitativamente.
Entregables y Documentación	15 Puntos	Configuración del repositorio privado agregando al auxiliar como colaborador. Redacción del Manual Técnico y de Usuario en formato Markdown. Diagrama de flujo de datos cross-port.

14.2 Sección de Funcionamiento del Sistema (Conocimiento) - 60 Puntos

Criterio Evaluado	Ponderación	Aspectos Técnicos de Calificación en C# y .NET Core
Módulo de Ingesta y Regex	10 Puntos	Procesamiento masivo de configuraciones mediante consultas estructuradas de XPath con <code>Xmldocument</code> . Validación rigurosa de datos con expresiones regulares y reversión atómica de transacciones.
Implementación de TDAs	15 Puntos	Funcionamiento libre de fallos de la Matriz Dispersa Ortogonal (plano espacial). Auto-balanceo del Árbol AVL para el catálogo global. Encolamiento en el ABB de prioridades y bitácora.
Reportes con Graphviz	15 Puntos	Compilación directa en memoria RAM del código de diagramación DOT y generación asíncrona de flujos vectoriales SVG sin escrituras en disco. Trazabilidad interactiva del buffer y enrutamiento.
Protocolo REST Multi-Instancia	15 Puntos	Despliegue concurrente en puertos locales 5000 y 5001. Comunicación síncrona de enrutamiento utilizando <code>HttpClient</code> autenticada de forma segura mediante HTTP Basic Authentication.
Internacionalización (i18n)	5 Puntos	Soporte de traducción dinámica multi-idioma (Español / Inglés) configurable desde la barra de navegación del sitio web.

14.3 Tabla Resumen de Penalizaciones Severas

Concepto de Incumplimiento del Estudiante	Impacto de Penalización sobre la Calificación Final
Plagio o Copia Estructural de Código (Entre grupos o internet)	-100% de la nota final (Asignación automática de 0 puntos y remisión a Dirección Académica).
Uso de código copiado de IA Generativa sin dominio técnico	-100% de la nota final (Anulación del proyecto en la defensa individual presencial).
Uso de colecciones dinámicas nativas de .NET para los TDAs	-75% de deducción directa sobre la nota del componente evaluado.
No registrar el mínimo de 3 commits semanales por integrante	-20% de penalización por hito en la nota de fase del estudiante inactivo.
No agregar al auxiliar de cátedra como colaborador del repositorio	-100% de la nota final (El proyecto no se recibe para calificación).
La aplicación no está desarrollada sobre el entorno C# / .NET Core	-100% de la nota final (Anulación inmediata por incumplimiento técnico).
Escritura física de archivos DOT o imágenes en disco duro	-30% de deducción sobre la calificación de la sección de visualización.
Entregas extemporáneas fuera de la plataforma académica oficial	-100% de la nota final (No se aceptan entregas por canales externos).

15. Anexo: Archivos de Configuración y Prueba Oficiales

Los siguientes archivos estructurados oficiales se proporcionan con el fin de evaluar y calificar de forma presencial e individual el comportamiento lógico de las estructuras de memoria RAM ante escenarios consistentes, de carga masiva y con errores controlados:

15.1 Archivo de Configuración 1: Escenario de Carga Exitosa (Constelación Norte - Puerto 5000)

Este archivo se cargará en la instancia que corre en el puerto local 5000, estableciendo la topología del norte y las antenas de recepción en Centroamérica:

```
<?xml version="1.0" encoding="utf-8"?>
<orbitnet>
  <constelaciones_ecuatoriales>
    <satelite id="SAT-ECU-0001">
      <nombre>Starlink-Norte-A</nombre>
      <enlace_ip>127.0.0.1</enlace_ip>
    </satelite>
    <satelite id="SAT-ECU-0002">
      <nombre>Starlink-Norte-B</nombre>
      <enlace_ip>127.0.0.1</enlace_ip>
    </satelite>
  </constelaciones_ecuatoriales>
</orbitnet>
```

```

<orbitas_polares>
  <polar id="POLAR-NORTE-S">
    <satelite id="SAT-POL-1001">
      <nombre>Polar-Norte-1</nombre>
      <frecuencia>450.15</frecuencia>
    </satelite>
    <satelite id="SAT-POL-1002">
      <nombre>Polar-Norte-2</nombre>
      <frecuencia>450.30</frecuencia>
    </satelite>
  </polar>
</orbitas_polares>
<antenas_terrestres>
  <antena id="ANT-GTM-401">
    <nombre>Estación Central USAC Edificio T3</nombre>
    <coordenadas>14.5891, -90.5514</coordenadas>
    <ip_nodo>10.0.0.50</ip_nodo>
  </antena>
</antenas_terrestres>
</orbitnet>

```

15.2 Archivo de Configuración 2: Escenario de Carga Exitosa (Constelación Sur - Puerto 5001)

Este archivo se cargará en la instancia del puerto local 5001, cubriendo las operaciones de telecomunicaciones del Hemisferio Sur:

```

<?xml version="1.0" encoding="utf-8"?>
<orbitnet>
  <constelaciones_ecuatoriales>
    <satelite id="SAT-ECU-0101">
      <nombre>Starlink-Sur-A</nombre>
      <enlace_ip>127.0.0.1</enlace_ip>
    </satelite>
    <satelite id="SAT-ECU-0102">
      <nombre>Starlink-Sur-B</nombre>
      <enlace_ip>127.0.0.1</enlace_ip>
    </satelite>
  </constelaciones_ecuatoriales>
  <orbitas_polares>
    <polar id="POLAR-SUR-S">
      <satelite id="SAT-POL-2001">
        <nombre>Polar-Sur-1</nombre>
        <frecuencia>480.15</frecuencia>
      </satelite>
      <satelite id="SAT-POL-2002">
        <nombre>Polar-Sur-2</nombre>
        <frecuencia>480.30</frecuencia>
      </satelite>
    </polar>
  </orbitas_polares>
  <antenas_terrestres>

```

```
<antena id="ANT-ARG-501">
  <nombre>Subestación Regional Argentina</nombre>
  <coordenadas>-34.6037,-58.3816</coordenadas>
  <ip_nodo>10.0.0.90</ip_nodo>
</antena>
</antenas_terrestres>
</orbitnet>
```

15.3 Archivo de Control: Escenario de Errores y Excepciones

Este archivo contiene fallos sintácticos deliberados destinados a auditar y verificar que el analizador web realice un rechazo transaccional atómico preventivo, salvaguardando el estado de la memoria RAM del servidor:

```
<?xml version="1.0" encoding="utf-8"?>
<orbitnet>
  <constelaciones_ecuatoriales>
    <satelite id="SAT-ECU-9001">
      <nombre>Satélite Guardián S1</nombre>
      <enlace_ip>127.0.0.1</enlace_ip>
    </satelite>
    <satelite id="SAT-ECU-99">
      <nombre>Satélite Defectuoso ID</nombre>
      <enlace_ip>127.0.0.1</enlace_ip>
    </satelite>
    <satelite id="SAT-ECU-9002">
      <nombre>Satélite Defectuoso IP</nombre>
      <enlace_ip>172.300.4.1</enlace_ip>
    </satelite>
  </constelaciones_ecuatoriales>
  <orbitas_polares>
    <polar id="POLAR-AUX">
      <satelite id="SAT-POL-AB12">
        <nombre>Satelite Polar Invalido</nombre>
        <frecuencia>400.0</frecuencia>
      </satelite>
      <satelite id="SAT-POL-8080">
        <nombre>Polar Respaldo OK</nombre>
        <frecuencia>415.5</frecuencia>
      </satelite>
    </polar>
  </orbitas_polares>
  <antenas_terrestres>
    <antena id="ANT-GTM-405">
      <nombre>Antena Falla Coordenadas</nombre>
      <coordenadas>14.5891,-90.XYZ55</coordenadas>
      <ip_nodo>10.0.0.1</ip_nodo>
    </antena>
    <antena id="ANT-GTM-406">
      <nombre>Antena Respaldo OK</nombre>
      <coordenadas>14.2012,-90.1114</coordenadas>
```

```
        <ip_nodo>10.0.0.2</ip_nodo>  
    </antena>  
</antenas_terrestres>  
</orbitnet>
```